Group Project: Generative-AI Embodied Robot Assistant

Perception, Planning and Interactions | *Webots R2025a* | ***25 marks*** | *Groups of 3-5*

Due by Sunday 11:30pm, 01 November 2026

1. Project Overview

In this project, your group will develop a conversational robot assistant in Webots R2025a. A user will describe a task in ordinary spoken language. Your system must interpret the request, use a genuine generative model to propose a high-level plan, validate the plan with deterministic Python code, and execute only approved robot skills. The completed system must be able to:

- accept and transcribe a spoken goal;
- ask for clarification when essential information is missing;
- generate an explicit, machine-readable plan;
- reject malformed, unsupported or unsafe plans before execution;
- execute approved actions using Webots motors and sensors;
- record what happened during each action; and
- complete the task, revise the remaining plan, or stop safely with a truthful explanation.

Required end-to-end outcome: the user states a goal; the system obtains any necessary clarification; the generative model proposes a symbolic plan; PlanValidator accepts or rejects the plan; TaskExecutive executes an accepted plan through RobotSkills; sensor evidence updates WorldState; and the system reports the final outcome or performs bounded recovery.

2. Relationship to the Subject

The project brings together topics from the whole subject, with particular emphasis on Weeks 8-12. Earlier robotics topics from Week 1-7 provide the physical capabilities required by the assistant, while later topics provide the main interaction, language and generative-AI components.

Subject area	Role	What students must demonstrate
Perception and vision	Supporting	Use camera evidence to search for simple coloured objects and distinguish found, not-found and uncertain outcomes.
Mapping, localisation and navigation	Supporting	Move between named locations using feedback from the supplied Webots environment.
Human-robot interaction	Major	Accept goals, request clarification, report progress and explain outcomes coherently.
Speech recognition and synthesis	Major	Receive spoken instructions and provide spoken robot responses.
Natural-language processing	Major	Ground different phrasings to valid goals, entities, constraints and conditions.
Generative AI	Major	Produce structured high-level plans and, where required, revised plans.

Marks are awarded for demonstrated behaviour and supporting evidence.

3. Supplied Webots Environment

You are not required to build the project from an empty folder. The supplied starter package contains a Webots world, an E-puck robot, named locations, a required red-ball search target, coloured distractor objects, a movable obstacle, a setup Supervisor for random target placement, configuration files, controller modules, a baseline symbolic-plan schema and initial validator tests. The robot controller is intentionally incomplete. Each TODO identifies functionality that your group must design, implement or justify.

- **Named locations:** reception, main corridor, Office A, Office B, laboratory and meeting room.
- Required search object: the assessed target is the red ball. A green ball and other coloured shapes may appear as distractors, so detecting any ball or any coloured region is not sufficient. Support for additional object types is optional.
- Randomised placement: at the start of every trial, including after a simulation reset or reload, the supplied setup Supervisor randomly places the red ball at one of the approved spawn points in the searchable named locations. The spawn points keep the ball on a valid surface and within the intended camera-search area. Groups must retain this randomisation and must not manually fix the target at a preferred location for assessment or reported evaluation.
- **Available evidence:** camera images, eight E-puck proximity sensors named ps0 to ps7, GPS and Compass.
- **Permitted localisation support:** the supplied coordinates and waypoint graph may be used with GPS and Compass to estimate the robot's position, heading and arrival.
- Supervisor boundary and prohibited shortcuts: the supplied setup Supervisor may move the red ball only to initialise a random trial and may log the selected spawn point for evaluation. The assessed robot controller must not access that value, query Supervisor fields, read object DEF names or translations, or otherwise obtain hidden ground truth. The robot must move using its wheel motors and must not be teleported by changing translation or rotation fields. It must not contain special-case responses for anticipated marker prompts or assume a predetermined target location or search outcome.
- **World compatibility:** you may extend the world or controller, but the required locations, devices, object classes and controller interfaces must continue to work in the supplied assessment world.

4. Required System Architecture

The starter code separates language generation from deterministic robot control. The following class names refer to the supplied Python files. You may refactor the code, but the same responsibilities and safety boundaries must remain clear and testable.

Supplied component	File	Responsibility
SpeechIO	*speech_io.py*	Recognise the user's speech and produce spoken responses.
DialogueManager	*nlu_dialogue.py*	Detect missing information, retain a pending goal and combine clarification answers with that goal.
LLMInterface	*llm_interface.py*	Hide model-specific code and return a parsed symbolic plan as ordinary Python data.
PlanValidator	*plan_validator.py*	Check the untrusted plan and return a Boolean result together with precise errors.
TaskExecutive	*task_executive.py*	Advance an accepted plan one step at a time without blocking the Webots loop.
RobotSkills	*robot_skills.py*	Map approved symbolic actions to navigation, perception, speech and memory operations.
Navigator / ObjectPerception	*navigation.py / perception.py*	Use Webots devices to perform movement and camera-based search.
WorldState	*world_state.py*	Store the current task, observations, outcomes and retry/replanning information.

The generative model may propose only high-level symbolic actions. It must not set wheel speeds, access Webots devices, execute generated Python or shell commands, modify *PlanValidator*, or decide that its own output is safe. Every initial or revised plan must pass through *PlanValidator*.

Speech recognition and model inference may be slow. Run these operations in a worker thread, process or equivalent asynchronous component, then return their results to the main controller as data. Only the Webots controller thread should access Webots devices. The simulation loop must remain responsive while planning or speech processing is active.

5. Functional Requirements

- **Natural interaction**. Accept a spoken goal, acknowledge it, ask a relevant clarification when required and speak the outcome. Typed input may be used for debugging but does not replace the assessed speech capability.
- **Language understanding**. Support different phrasings and common synonyms instead of matching only a fixed command sentence.
- **Structured planning.** Use a genuine generative model to convert the goal into an explicit plan that follows the symbolic-plan contract.
- **Plan validation**. Check every initial and revised plan before execution. Reject unsupported actions, invalid entities, missing arguments, incorrect data types and malformed structures.
- **Grounded execution.** Execute only documented RobotSkills and determine action outcomes from Webots sensors and controller state, not from claims made by the LLM.
- Randomised red-ball search. Locate and identify the red ball after the setup Supervisor has independently selected its position. The system must search using camera and navigation evidence, distinguish the green ball and other distractors, and report FOUND, NOT_FOUND or UNCERTAIN truthfully.
- **Execution monitoring**. Record whether each action is running, succeeds, fails, times out, becomes blocked or is cancelled.
- **Conditional behaviour**. Use values stored in *WorldState* to select a previously validated then/else branch.
- **Replanning**. When a failure or new observation makes the remaining plan unsuitable, generate a meaningfully revised remainder, validate it again and enforce a retry limit.
- **Responsiveness** and **cancellation**. Allow the active task to be stopped safely and keep the simulation responsive during speech, model inference and long-running robot actions.
- **Reproducibility**. Provide runnable code, configuration instructions, tests, evaluation records and a demonstration-ready setup.

6. Language and Planning Capability

The system must handle the following types of user request. Examples illustrate the required behaviour; students must also test different wording that is not copied from these examples.

Request type	Example	Required system behaviour
Direct	*"Go to the laboratory."*	Generate and execute one valid navigation action.
Sequential	*"Go to the laboratory, then return to reception."*	Preserve and execute the requested order.
Search	*"Find the red ball."*	Ground the required target as the red ball, search candidate locations using the camera, reject distractors and report an evidence-based outcome. The target location must not be assumed from a previous trial.
Conditional	*"Look in the lab. If the red ball is not there, check the meeting room."*	Represent both branches and select one using the actual search result.

Request type	Example	Required system behaviour
Constrained order	*“Check the lab first; only search Office A if the ball is absent.”*	Preserve both the ordering and the condition.
Ambiguous	*“Find the ball.”*	Ask which ball or colour is intended unless the dialogue context already makes the required red-ball target unambiguous.
Unsupported	*“Fly through the wall to Office B.”*	Reject the unsupported action or ask for an executable reformulation before actuation.
Status or explanation	*“What are you doing and why?”*	Answer from the current goal, plan step and evidence stored in WorldState.

7. Symbolic Plan Contract

The supplied version 1.0 contract defines the format exchanged between *LLMInterface*, *PlanValidator* and *TaskExecutive*. LLMInterface should return the parsed plan as a Python dictionary. Groups may extend the contract only when the extension remains explicit, deterministic to validate and compatible with all required behaviours. Document every extension and add corresponding validator tests.

```
{
  "version": "1.0",
  "goal": "find the red ball; check the laboratory first",
  "steps": [
    {"action": "navigate", "location": "laboratory"},
    {"action": "search", "object": "ball", "colour": "red"},
    {
      "action": "conditional",
      "condition": {
        "source": "last_search_result",
        "operator": "found",
        "value": false
      },
      "then": [
        {"action": "navigate", "location": "meeting_room"},
        {"action": "search", "object": "ball", "colour": "red"}
      ],
      "else": []
    },
    {"action": "return_home"},
    {"action": "report", "message": "Report the evidence-based result."}
  ]
}
```

Action	Required arguments	Meaning and expected outcome
navigate	location	Move to a configured location. Report SUCCEEDED only after the arrival criterion is met; otherwise report FAILED, TIMEOUT or BLOCKED with evidence.
search	object; optional colour	Perform a bounded camera search and produce FOUND, NOT_FOUND or UNCERTAIN evidence.
observe	none	Update relevant WorldState evidence without claiming that the overall task has succeeded.
ask	question	Ask the user for information and place the dialogue in a waiting-for-answer state.
report	message	Speak or display a message that is consistent with the current task state; no robot motion is performed.
return_home	none	Use the navigation skill to return to reception.
stop	none	Stop motion and end or cancel the active task safely.
conditional	condition, then, else	Select one already validated branch by evaluating the permitted condition against WorldState.

8. Plan Validation and Safety

Every plan generated by an LLM must be treated as untrusted data. The supplied *PlanValidator* class, located in *plan_validator.py*, checks a generated plan before the robot is allowed to execute it. The method *PlanValidator.validate(plan)* returns two values: "*valid*" is *True* when all checks pass and *False* otherwise; and "*errors*" is a list describing each detected problem. If valid is False, the plan must be rejected and the robot must remain stopped. Only a valid plan may be passed to the TaskExecutive class in *task_executive.py*.

At minimum, plan validation must check:

- **Overall structure:** the plan is a JSON object containing the correct version, a non-empty goal and a non-empty list of steps.
- **Size limits:** the plan does not exceed the permitted number of steps, nesting depth or generated-text length.
- **Permitted actions:** every action appears in the approved action list. Unknown, misspelled or unsupported actions are rejected.
- **Action parameters:** each action contains all required parameters, uses the correct data types and satisfies any defined value limits.
- **Known values:** locations, objects, colours, condition sources and condition operators match the project configuration.
- **Conditional branches:** each conditional contains valid then and else lists. Conditions use only predefined grammar and are never evaluated with Python *eval()* or generated executable code.
- **Semantic safety:** the plan does not contain contradictory steps, excessive repeated navigation, unlimited retries or other unsafe or physically impossible behaviour.
- **Prohibited content:** the plan contains no direct motor commands, Python or shell commands, file or network operations, arbitrary tool calls or other unapproved executable content.

When validation fails, the system may ask the user to clarify the goal, request a corrected plan subject to a defined retry limit, or report that the goal cannot be represented safely. Any corrected or replanned output is a new untrusted plan and must pass PlanValidator again before execution.

9. Task Executive and Robot Skills

The supplied *TaskExecutive* class in *task_executive.py* manages a plan that has already passed validation. Its *start(plan)* method accepts the plan, while *update()* advances at most one small unit of work during each Webots controller step. It must not call slow blocking functions or run an entire plan in one update.

RobotSkills in *robot_skills.py* translates each approved symbolic action into the appropriate navigation, perception, speech or memory operation. Each long-running skill should provide *start(step), update()* and *cancel()*, or an equivalent non-blocking interface. The supplied implementations are incomplete and must be extended.

State or outcome	Meaning	Required response
IDLE	No accepted task is active.	Keep the motors stopped and allow a new goal.
RUNNING	A plan or robot skill is still active.	Perform one bounded update and retain current-step evidence.
SUCCEEDED	The skill's measurable completion criterion has been met.	Record the evidence, update WorldState and advance to the next step.
FAILED	The skill cannot meet its completion criterion.	Stop the skill, record the reason and enter the recovery decision.
TIMEOUT or BLOCKED	A progress limit was exceeded or the route is obstructed.	Stop motion, record sensor/progress evidence and recover or fail safely.
CANCELLED	The user or system cancelled the task.	Stop all motion and clear or retain pending work according to the documented policy.

Every motion skill must define a measurable completion condition, a timeout or progress limit, a speed limit and a cancellation path. An LLM statement such as 'navigation succeeded' is not evidence that the robot arrived.

10. Navigation, Perception and Evidence

Navigation and perception provide the physical evidence used by *TaskExecutive* and *WorldState*. These components must use current Webots device readings rather than expected outcomes encoded during development.

Each assessed search trial begins with a simulation restart or reset. The setup Supervisor then chooses a red-ball spawn point independently of the robot controller. The robot must discover the target from permitted sensors; information printed or stored by the Supervisor is evaluation ground truth and must not be passed to the planner, WorldState, perception module or navigation module.

- **Navigation:** use wheel motors together with repeated GPS and Compass feedback. Define the arrival tolerance, heading control, speed limits, waypoint progress, obstacle threshold and timeout or blocked-route outcome.
- **Proximity safety:** ps0 to ps7 are the eight built-in E-puck proximity sensors arranged around the robot. The supplied Navigator treats ps0, ps1, ps6 and ps7 as front obstacle sensors. If reverse motion is implemented, monitor the sensors facing the direction of travel. Stop the motors before beginning any recovery action.
- **Perception**: analyse current camera frames to identify the red ball and reject the green ball and other distractors. A robust colour-and-shape method is sufficient; a trained detector is not required. Document the colour space, segmentation method, minimum area or circularity evidence, distractor handling and confidence rule.
- **Bounded search**: visit or inspect candidate locations using a documented finite policy. At each location, examine a defined sequence of views or rotations for a finite number of frames or seconds. FOUND requires positive red-ball evidence; NOT_FOUND requires completion of the stated search policy; UNCERTAIN means the available evidence is insufficient. A failed detection must not be reported as proof that the ball is absent from an unsearched location.
- **No hidden ground truth**: only the setup Supervisor may know the random spawn point. The assessed controller must not query Supervisor fields, object translation values, DEF names, shared files or messages to determine the target's identity or location. Its result must be derived from permitted robot sensors and recorded execution evidence.

11. LLM Choice and Demonstration Readiness

Groups may use the Lab 5 Qwen GGUF model or another local/online model. Model choice is flexible. Marks depend on the demonstrated behaviour of the complete system only.

- **Model files:** do not submit GGUF files, downloaded model weights or model caches.
- **Credentials:** do not place API keys, passwords, tokens or service-account files in the source code or submission. Read online credentials from environment variables or secure local configuration excluded from the submitted archive.
- **Model isolation:** keep backend-specific imports, configuration and response parsing inside LLMInterface so that the planner can be tested separately from validation and robot execution.
- **Reproducibility:** record the exact model identifier, runtime/client version, relevant generation settings and configuration procedure in the README and report.
- **Test fixtures:** use deterministic saved responses or a mock backend for software tests where useful, but label them clearly and do not present them as genuine model-performance evidence.

Groups using the Lab 5 Qwen model do not need to submit its weights or repeat the Lab 5 installation instructions. Groups using another local or online LLM must provide a fully configured, demonstration-ready system. During

marking, markers will not download model files, create accounts, supply credentials, approve paid usage, edit source code or troubleshoot model-specific installation problems.

12. Recommended Development Sequence

Develop and test the deterministic components before connecting them to live speech and a live language model.

1. Open the supplied Webots world; confirm all devices; and verify across several resets that the setup Supervisor places the red ball at different approved spawn points without exposing the chosen point to the robot controller.
2. Complete and extend *PlanValidator*, then make all validator tests pass using manually prepared plan dictionaries.
3. Implement *WorldState*, condition evaluation and the non-blocking *TaskExecutive* using deterministic plan fixtures.
4. Implement and test navigation, obstacle stopping, red-ball perception, distractor rejection, bounded multi-location search and cancellation as independent RobotSkills.
5. Implement *LLMInterface* for the selected backend and retain the raw response, parsed plan and parsing errors.
6. Add *DialogueManager* and *SpeechIO*, including clarification handling and the policy for a new goal received while the robot is busy.
7. Integrate one validated conditional path and one bounded replanning path.
8. Run the full evaluation protocol across independent randomised restarts, analyse failures and freeze a demonstration-ready configuration before submission.

13. Assessment Criteria (25 marks)

Criterion	Marks	What will be assessed
Speech and natural-language interaction	3	Usable spoken interaction, varied language, clarification and appropriate responses.
Generative-AI goal interpretation	3	Correct interpretation of goals, constraints, conditions and relevant entities.
Structured task-plan generation	3	Explicit, meaningful and executable plans, including sequences and conditions.
Plan validation and grounding	3	Reliable schema/action/location checks and strict separation between LLM reasoning and robot actuation.
Execution monitoring and replanning	3	Detects failed/changed conditions and adapts rather than blindly executing the original plan.
Perception and navigation integration	3	Robot skills provide sufficiently reliable perception, search and navigation to support the task.
Robustness, failure handling and explanation	2	Graceful handling of ambiguity, impossible requests, uncertainty and explanatory status.
Demonstration, technical understanding, and technical report	5	Systematic demonstration and evidence that the understanding of the integrated solution in the technical report.

14. Submission Guidelines

Filename

- Each group must submit one compressed ZIP file through the project submission link on Moodle.
- Name the submitted file using the following convention: "*CSCI444_944_GroupXX_Project.zip*". Replace "*XX*" with your group number.
- For example: "*CSCI444_944_Group07_Project.zip*"
- Only one group member should upload the final ZIP file on behalf of the group.

Required contents

The ZIP file must contain:

- **Complete Webots project**: Include the runnable *Project_GenAI_Embodied_Assistant* folder, including the setup Supervisor and its approved red-ball spawn-point configuration. Do not include a version that fixes the ball at one location. Do not include the LLM model in the ZIP file. If you use an LLM different from the one used in Lab 5, include a download link in the *requirements.txt* file.
- **Dependencies**: Include updated *requirements.txt* specifying the Python libraries or models required by the implementation.
- **README**: Include clear instructions describing the required software versions, selected LLM, audio-device configuration, model configuration, starting Webots world and controller, and commands for running and testing the system.
- **Technical report:** Include the final report as *GroupXX_Project_Report.pdf*, replacing *XX* with the Moodle group number. The report must follow the requirements below.

Technical report requirements

The report must be a single PDF of **no more than 10 A4 pages**, excluding the title page, reference list and appendices. Appendices are supporting material only. Important design decisions, results and analysis must appear in the main report.

- **Group contributions and references:** state each member's substantive contributions and identify any reused code, models, datasets, libraries or external sources using appropriate citations.
- **Project summary and task definition:** outline the intended user experience, required red-ball search task, supported request types, assumptions and the system's success criteria.
- **System architecture:** provide a diagram and explain the data and control flow among speech input, *DialogueManager, LLMInterface, PlanValidator, TaskExecutive, RobotSkills, WorldState* and Webots devices.
- **Language interaction and generative planning:** identify the model and important generation settings; explain prompt design, clarification handling and structured-plan parsing; and include representative examples of an accepted plan, a conditional plan and a rejected or corrected model response. Do not include credentials or model weights.
- **Plan validation and safety:** describe the plan contract, allow-list, parameter and semantic checks, limits, condition grammar, cancellation and bounded replanning policy. Explain how the setup *Supervisor* is isolated from the assessed controller and provide tests for valid, malformed, unsupported and unsafe plans.
- **Robot execution, navigation and perception:** describe the non-blocking executive, completion and timeout rules, waypoint and heading control, proximity-sensor safety, red-ball detection method, distractor rejection, confidence rule and bounded multi-location search policy. State and justify the principal thresholds and tolerances.
- **Results and critical analysis:** present aggregate results and representative traces for plan validity, task completion, search accuracy, distractor rejection, navigation outcomes, completion time and recovery behaviour. Analyse important failures and explain what the evidence shows; do not report only successful demonstrations.
- **Limitations, ethics and future improvements:** discuss known limitations, uncertainty, privacy or service risks associated with speech and online models, and realistic improvements.
- **Do not directly copy and paste code snippets into the technical report:** this will result in a significant deduction of marks.

15. Group Work and Individual Understanding

The project is submitted as a group system, but every student is expected to understand the complete architecture. During demonstration, the marker may direct technical questions to any group member and may ask a member to explain a component other than the one they primarily implemented.

*****END*****